41039 Programming 1

Revision of Week 10: writing files and reliable reports

Student copy: questions only



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Today's revision: 10 questions, about 35 minutes

#	Type	Topic
Q1	Fill in the blanks	Newlines, modes, closing, quoting
Q2	Multiple choice	Keep the old line, add one (Java)
Q3	Multiple choice	`with` and an exception (Python)
Q4	Predict the output	`print`, `println`, append (Java)
Q5	Predict the output	A name with a comma (Python)
Q6	Find the bug	The report file (Java, 4 bugs)
Q7	Find the bug	Saving twice (Python, 3 bugs)
Q8	Write the code	Report rows, no file needed (Java)
Q9	Write the code	A log that only grows (Python)
Q10	Write the code	One report, any writer (Java)

How it works

1. Read the question and answer it **alone** first. Write your answer down.
2. Then compare with a neighbour.
3. We go through the answers in the lecture.

Week 10 in five ideas (read this first)

1. **Output is a contract.** Fix the field order, the delimiter and the line ending. `println` ends the line; `print` and Python's `write` add **nothing**.
2. **Overwrite or append.** `"w"` and `new PrintWriter(name)` **empty** an existing file. `"a"` and `new FileWriter(name, true)` add to the end.
3. **Close it.** Output waits in a buffer until it is flushed. try-with-resources and `with` close (and flush) the file, even on an exception. `PrintWriter`'s `println` never throws: ask `checkError()`.
4. **Delimiters need a rule.** A comma *inside* a name breaks `split(",")`. Python's `csv` module quotes such a field and reads it back whole. Everything read back is **text**.
5. **Calculate, format, write: three jobs.** A method that *returns* the text can be tested in memory (`StringWriter`, `StringIO`) without a real file.

```java
try (PrintWriter out = new PrintWriter(
        new FileWriter("r.csv", true))) {
    out.println("Linh,85");  // ends the line
}  // closed and flushed here
```

```python
with open("r.csv", "a") as f:
    f.write("Linh,85\n")  # you add \n
```

Q1 Fill in the blanks Newlines, modes, closing, quoting

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- Python's `f.write(s)` writes exactly `s`. Unlike Java's `println`, it adds no **(1)** .
- Opening a file with mode `"w"` **(2)** it first. Mode **(3)** keeps the old lines and writes after them.
- Output can wait in a **(4)** . `try`-with-resources and `with` **(5)** the file for you, even after an exception.
- A comma inside a name would cut the field in two, so a CSV writer puts that field in **(6)** .

Word bank (2 min, alone)

a) newline	**f)** quotes
b) empties	**g)** `"r"`
c) `"a"`	**h)** header
d) buffer	**i)** checkError
e) close	**j)** appends

Q2 Multiple choice Keep the old line, add a new one

Goal: `s.txt` holds one line, `old`. Afterwards it must hold **two** lines: `old`, then `new`.

```
// import java.io.*;
// main throws IOException
try (PrintWriter out =
         new PrintWriter(____)) {
    out.println("new");
}
```

Which fills the blank `____`? (90 seconds)

A. `"s.txt"`

B. `new FileWriter("s.txt")`

C. `new FileWriter("s.txt", true)`

D. `new StringWriter()`

Hint: all four compile and run. For each one, ask: what is in `s.txt` afterwards?

Q3 Multiple choice `with` and an exception

```
try:
    with open("r.txt", "w") as f:
        f.write("a\n")
        n = int("two")
        f.write("b\n")
except ValueError:
    print("closed?", f.closed)
with open("r.txt") as f:
    print(repr(f.read()))
```

What does it print? (60 seconds)

A. `closed? False` then `''`
B. `closed? True` then `'a\n'`
C. `closed? True` then `'a\nb\n'`
D. `closed? False` then `'a\n'`

`int("two")` raises a `ValueError`. `repr` shows the text with quotes, and a newline as `\n`.

Q4 Predict the output `print`, `println` and append

```
// in main (throws IOException); import java.io.*;
try (PrintWriter out = new PrintWriter("log.txt")) {
    out.print("A");
    out.println("B");
    out.print("C");
}
try (PrintWriter out = new PrintWriter(
        new FileWriter("log.txt", true))) {
    out.println("D");
}
BufferedReader in = new BufferedReader(
    new FileReader("log.txt"));
String line;
int n = 0;
while ((line = in.readLine()) != null) {
    n++;
    System.out.println(n + ": " + line);
}
in.close();
```

Predict (90 seconds, alone)

1. Write down every line that is printed.

2. What is printed if the second writer is `new PrintWriter("log.txt")` instead?

Q5 Predict the output A name with a comma

```
import csv
with open("marks.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["name", "mark"])
    w.writerow(["Le, An", 85])
    w.writerow(["Binh", 49])
with open("marks.csv", newline="") as f:
    lines = f.read().splitlines()
print(lines[1])
print(len(lines[1].split(",")))
with open("marks.csv", newline="") as f:
    rows = list(csv.reader(f))
print(rows[1][0])
print(rows[2])
```

`splitlines()` gives the file's lines without their line endings. `csv.reader` gives one list of fields per line.

Predict (90 seconds, alone)

1. Write down the **four** lines that are printed.

2. You want Binh's mark as a number you can add up. What do you write?

Q6 Find the bug The report file (Java)

```java
import java.io.*;
public class Report {
    // header, a line per student, pass count
    static void save(String f, String[] names,
                     int[] marks) {
        FileWriter fw = new FileWriter(f);
        PrintWriter out = new PrintWriter(fw);
        out.println("name,mark");
        int passed = 0;
        for (int i = 0; i < names.length; i++) {
            out.print(names[i] + "," + marks[i]);
            if (marks[i] > 50) passed++;
        }
        out.println("passed," + passed);
    }
}
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

What the method should do

- Write `name,mark`, one `name,mark` line per student, then `passed,<count>`
- A mark of 50 or more is a pass
- When save returns, every line is in the file

Test

```java
// main (throws IOException) is in class Demo
String[] names = {"An", "Binh", "Chi"};
int[] marks = {50, 49, 90};
Report.save("r.csv", names, marks);
// then print every line of r.csv
```

Expected output

```
name,mark
An,50
Binh,49
Chi,90
passed,2
```

Q7 Find the bug Saving twice (Python)

```python
def save(path, marks):
    with open(path, "a") as f:
        f.write("name,mark")
        for name, mark in marks.items():
            f.write(f"{name},{mark}\n")

def total(path):
    with open(path) as f:
        lines = f.read().splitlines()
    result = 0
    for line in lines[1:]:
        result += line.split(",")[1]
    return result

marks = {"An": 85, "Binh": 49}
save("m.csv", marks)
save("m.csv", marks)
with open("m.csv") as f:
    print(f.read(), end="")
print(total("m.csv"))
```

✗ **Find 3 bugs** in lines 1–13 (3 min). Give each line number and its fix.

What the code should do

- Saving again **replaces** the old report
- The header is a line of its own
- `total` returns the sum, as a number

Expected	✗ Actual

`name,mark`
`An,85`
`Binh,49`
`134`
	`name,markAn,85`
	`Binh,49`
	`name,markAn,85`
	`Binh,49`

… then it crashes at line 12 with

```
TypeError: unsupported operand type(s) for +=: 'int' and 'str'
```

Q8 Write the code Report rows, no file needed (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `row`: the name, the mark and `pass` or `fail`, joined by commas. A mark of 50 or more is a pass.
- `summary`: `"<passed>/<count> passed"`, or `"no marks"` when the array is empty.
- Both **return** the text and print nothing: the caller decides where it goes (screen, file or test).

Skeleton

```
class Rows {
    // "An,50,pass" or "Binh,49,fail"
    static String row(String name, int mark) {
        return "";  // TODO
    }
    // "2/3 passed", or "no marks" if empty
    static String summary(int[] marks) {
        return "";  // TODO
    }
}
```

Test (in `main`)

```
System.out.println(Rows.row("An", 50));
System.out.println(Rows.row("Binh", 49));
int[] m = {50, 49, 90};
System.out.println(Rows.summary(m));
System.out.println(Rows.summary(new int[0]));
```

Must print

```
An,50,pass
Binh,49,fail
2/3 passed
no marks
```

Q9 Write the code A log that only grows (Python)

Finish the TODOs (alone, 4 min). `app.log` does not exist at the start. Use `with`.

- `log(path, message)`: add `message` as a new **last** line, keeping the earlier lines.
- `count_lines(path)`: how many lines the file has, or `0` if it does not exist (do not crash).

Skeleton

```python
def log(path, message):
    pass  # TODO

def count_lines(path):
    # TODO
    return 0
```

Test

```python
log("app.log", "start")
log("app.log", "save")
print(count_lines("app.log"))
log("app.log", "stop")
print(count_lines("app.log"))
print(count_lines("none.log"))
```

Must print

```
2
3
0
```

Q10 Write the code One report, any writer (Java)

Finish the `TODO` (alone, 4 min). Write `name,mark`, each row, then `count,<rows>`.

- Return `true` only if every write worked: `println` never throws, so ask `out.checkError()`.
- Do **not** close `out`: whoever opened it closes it.

Skeleton

```
class Report {
    // header, each row, then the count
    static boolean write(PrintWriter out,
                         String[] rows) {
        // TODO
        return false;
    }
}
```

Test (in `main`; `import java.io.*;`)

```
StringWriter mem = new StringWriter();
PrintWriter out = new PrintWriter(mem);
String[] rows = {"An,85", "Binh,49"};
System.out.println(Report.write(out, rows));
System.out.print(mem);
out.close();
System.out.println(Report.write(out, rows));
```

Must print

```
true
name,mark
An,85
Binh,49
count,2
false
```

Looking ahead: one method, three jobs

Last week: one method does it all

```
class ReportJob {
    void run(int[] marks) throws IOException {
        int passed = 0;
        for (int m : marks) if (m >= 50) passed++;
        try (PrintWriter out =
                 new PrintWriter("r.csv")) {
            for (int m : marks) out.println(m);
            out.println("passed," + passed);
        }
    }
}
```

Think (1 minute)

The pass mark changes to 40, and the office also wants the report as JSON. Where does each change go?

You should now be able to...

Writing files

- ✓ choose overwrite (`"w"`) or append (`"a"`, `true`) on purpose (Q1, Q2, Q4, Q7, Q9)
- ✓ end every line: `println`, or `"\n"` with `write` (Q1, Q4, Q6, Q7)
- ✓ close with `try`-with-resources or `with`, even on an exception (Q1, Q3, Q6)
- ✓ handle errors: `throws IOException`, `FileNotFoundError`, `checkError()` (Q6, Q9, Q10)

Reliable reports

- ✓ let `csv` quote a field with a comma; read it back with `csv.reader` (Q1, Q5)
- ✓ turn text read back into numbers: `int(...)` (Q5, Q7)
- ✓ keep calculating and formatting apart from the destination (Q8, Q10)
- ✓ test a report in memory, with the boundary and empty input (Q8, Q10)

Next: Week 11 – Designing object-oriented programs

Responsibilities, information experts, interfaces, substitution, cohesion and coupling.